

```

#include <stdio.h>
#include <stdlib.h>
#define MAX 100

// Maximum size of the queue
// Queue structure definition struct Queue
{
int arr[MAX];
int front;
int rear;
};
// Function prototypes
void initializeQueue(struct Queue *queue);
int isFull(struct Queue *queue);
int isEmpty(struct Queue *queue);
void enqueue(struct Queue *queue, int element); int dequeue(struct Queue *queue);

void displayQueue(struct Queue *queue);
int main()
{
struct Queue queue; int choice, element;
initializeQueue(&queue);
while (1)
{
printf("\nQueue Operations Menu:\n")
printf("1. Enqueue\n");
printf("2. Dequeue\n");
printf("3. Display\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice)
{
case 1:
printf("Enter element to enqueue: "); scanf("%d", &element);
enqueue(&queue, element);
break;
case 2:
element = dequeue(&queue);
if (element != -1)
{
printf("Dequeued element: %d\n", element); break;
}
case 3: displayQueue(&queue);
break;

```

```

case 4:
exit(0);
default:
printf("Invalid choice! Please enter a valid option.\n");
}
}
return 0;
}
// Function to initialize the queue
void initializeQueue(struct Queue *queue) { queue->front = -1;
queue->rear = -1;
}
// Function to check if the queue is full
int isFull(struct Queue *queue)
{
return queue->rear == MAX - 1;
}
// Function to check if the queue is empty
int isEmpty(struct Queue *queue)
{
return queue->front == -1 || queue->front > queue->rear;
}
// Function to enqueue an element to the queue void enqueue(struct Queue *queue, int
element)
{
if (isFull(queue))
{
printf("Error: Queue overflow. Cannot enqueue element.\n");
return;
}
if (isEmpty(queue))
{
queue->front = 0;
}
queue->arr[++queue->rear] = element; printf("Element enqueued: %d\n", element);
}
// Function to dequeue an element from the queue int dequeue(struct Queue *queue)
{
if (isEmpty(queue))
{
printf("Error: Queue underflow. Cannot dequeue element.\n");
return -1;
}
return queue->arr[queue->front++];
}

```

```
}  
// Function to display the queue elements  
void displayQueue(struct Queue *queue)  
{  
    if (isEmpty(queue))  
    {  
        printf("Queue is empty.\n");  
        return;  
    }  
    printf("Queue elements: ");  
    for (int i = queue->front; i <= queue->rear; i++)  
    {  
        printf("%d ", queue->arr[i]);  
    }  
    printf("\n");  
}
```